

BRIGHTHEARN EXERCISE 2

SQL Aggregate Functions & Operations

Q1, list all distinct departments in the Students table

```
SELECT DISTINCT department  
FROM Students;
```

Students
department
IT
HR
Finance

Q2, Get the average age of Students per department.

```
SELECT department, AVG(age) AS avg-age  
FROM Students  
GROUP BY department;
```

Students	
department	avg-age
IT	20
HR	22
Finance	23

Q3, Show departments with more than one Students

```
SELECT department, COUNT(*) AS Student_Count
FROM Students
GROUP BY department
HAVING COUNT(*) > 1;
```

Students	
department	Student_Count
IT	2
HR	2

Q4, Get all Students whose age is between 21 and 23.

```
SELECT Student_Id, name, age, department
FROM Students
WHERE age BETWEEN 21 and 23;
```

Students			
Student Id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

Q5, List all Students in the IT or HR department who are older than 21

```
SELECT Student_Id, name, age, department
FROM Students
```


WHERE department IN ('IT', 'HR')
AND age > 21;

Students			
Student_id	Name	age	department
2	Bob	22	HR
5	Eve	22	HR

Ob. Show total Credits per department, only
for departments with more than 5 total
Credits.

SELECT departments, SUM(Credits) AS totalCredits
FROM Courses
GROUP BY department
HAVING SUM(Credits) > 5;

Courses	
department	total-Credits
IT	11

OT, list all Courses that do not have
4 Credits.

SELECT Course-id, Course-name, department, Credits
FROM Courses
WHERE Credits < 4;

Courses			
Course-Id	Course-name	Department	Credits
101	SQL Basics	IT	3
104	Excel	Finance	2
105	Statistics	HE	3

Q8. Show the top 3 Courses by credits in descending order

```

SELECT Course-Id, Course-name, Credits
FROM Courses
ORDER BY Credits DESC
LIMIT 3;

```

Courses		
Course-Id	Course-name	Credits
103	Data Science	4
102	Python	4
101	SQL Basics	3

Q9. Get maximum, minimum, and average grade across all enrollments.

```

SELECT MAX(grade) AS max-grade
       MIN(grade) AS min-grade
       AVG(grade) AS avg-grade
FROM enrollments;

```

enrollments		
max-grade	min-grade	avg-grade
90	78	84

10. Count how many enrollments exists per Course.

```
SELECT Course-id, Count(*) AS enrollment-count
FROM enrollments
GROUP BY course-id.
```

Course-id	enrollment count
101	1
102	1
103	1
104	1
105	1

11. Find total salary and total bonus per department

```
SELECT department,
       SUM(salary) AS total-salary
       SUM(bonus) AS total-bonus
FROM Salaries
GROUP BY department
```


Salaries		
department	total Salary	total bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

12. Show departments where average salary is about 85000

```
SELECT department, AVG(salary) AS avg-salary
FROM Salaries
GROUP BY department
HAVING AVG(salary) > 85000
```

Salaries	
departments	avg-salaries
IT	61000
Finance	70000

13. List employees whose salary plus bonus is greater than 60000.

```
SELECT employee_id, name, salary, bonus
      (salary + bonus) AS total_compensation
FROM Salaries
WHERE (salary + bonus) > 60000
```

Salaries				
employee-id	name	Salary	bonus	total compensation
1	Tom	60000	5000	65000
3	Spite	70000	6000	76000
4	Tyke	62000	5500	67500

14. Show total and average budget per department.
Only include departments with average budget above 70000

```

SELECT department
      SUM(budget) AS total-budget,
      SUM(budget) AS
      AVG(budget) AS average-budget
FROM Projects
GROUP BY departments
HAVING AVG(budget) > 70000;

```

Projects		
department	total-budget	avg-budget
IT	270000	145000
Finance	80000	80000

15. List all projects with budgets between 80000 and 100000, excluding the Marketing department

```

SELECT Project-id, Project-name, department,
       budget
FROM Projects

```


WHERE budget BETWEEN 50000 AND 120000
AND department <> 'Marketing';

Projects			
Project-Id	Project-name	department	budget
1	AI APP	IT	120000
2	Payroll Systems	Finance	80000
3	Dashboard	IT	850000